



## .vscode\PythonImportHelper-v2-Completion.json

```
[ {
    "label": "numpy",
    "kind": 6,
    "isExtraImport": true,
    "importPath": "numpy",
    "description": "numpy",
    "detail": "numpy",
    "documentation": {} },
    {
    "label": "matplotlib.pyplot",
    "kind": 6,
    "isExtraImport": true,
    "importPath": "matplotlib.pyplot",
    "description": "matplotlib.pyplot",
    "detail": "matplotlib.pyplot",
    "documentation": {} },
    {
    "label": "FuncAnimation",
    "importPath": "matplotlib.animation",
    "description": "matplotlib.animation",
    "isExtraImport": true,
    "detail": "matplotlib.animation",
    "documentation": {} },
    {
    "label": "comet",
    "kind": 2,
    "importPath": "Examples.mycomet",
    "description": "Examples.mycomet",
    "peekOfCode": "def comet(x, y, *args, **kwargs):\n    tf = 3\n    fps=30\n    fig, ax = plt.subplots()\n    ln, = plt.plot([], [], 'r.-', alpha=0.5, *args, **kwargs)\n    dot, = plt.plot([], [], marker='o', markersize=10)\n    plt.xlabel('x')\n    plt.ylabel('y')\n    plt.grid()\n    def init():",
    "detail": "Examples.mycomet",
    "documentation": {} },
    {
    "label": "n",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "n = 50\n#Calc the n of unity\nk_values = np.arange(n)\nangles = 2 * np.pi * k_values / n\nroots = np.exp(1j * angles)\n#Create the figure\nplt.figure(figsize = (10,10))\nplt.plot(roots.real, roots.imag, 'o')\nplt.title('Roots of Unity of n=50')\nplt.xlabel('Real Part')",
    "detail": "HW.HW01_301",
    "documentation": {} },
    {
    "label": "k_values",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "k_values = np.arange(n)\nangles = 2 * np.pi * k_values / n\nroots = np.exp(1j * angles)\n#Create the figure\nplt.figure(figsize = (10,10))\nplt.plot(roots.real, roots.imag, 'o')\nplt.title('Roots of Unity of n=50')\nplt.xlabel('Real Part')\nplt.ylabel('Imaginary Part')\nplt.axhline(0, color='black',linewidth=0.5, ls='--')",
    "detail": "HW.HW01_301",
    "documentation": {} },
    {
    "label": "angles",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "angles = 2 * np.pi * k_values / n\nroots = np.exp(1j * angles)\n#Create the figure\nplt.figure(figsize = (10,10))\nplt.plot(roots.real, roots.imag, 'o')\nplt.title('Roots of Unity of n=50')\nplt.xlabel('Real Part')\nplt.ylabel('Imaginary Part')\nplt.axhline(0, color='black',linewidth=0.5, ls='--')\nplt.axvline(0, color='black',linewidth=0.5, ls='--')",
    "detail": "HW.HW01_301",
    "documentation": {} },
    {
    "label": "roots",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "roots = np.exp(1j * angles)\n#Create the figure\nplt.figure(figsize = (10,10))\nplt.plot(roots.real, roots.imag, 'o')\nplt.title('Roots of Unity of n=50')\nplt.xlabel('Real Part')\nplt.ylabel('Imaginary Part')\nplt.axhline(0, color='black',linewidth=0.5, ls='--')\nplt.axvline(0, color='black',linewidth=0.5, ls='--')\nplt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)",
    "detail": "HW.HW01_301",
    "documentation": {} },
    {
    "label": "plt.figure(figsize",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "plt.figure(figsize = (10,10))\nplt.plot(roots.real, roots.imag, 'o')\nplt.title('Roots of Unity of n=50')\nplt.xlabel('Real Part')\nplt.ylabel('Imaginary Part')\nplt.axhline(0, color='black',linewidth=0.5, ls='--')\nplt.axvline(0, color='black',linewidth=0.5, ls='--')\nplt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)\nplt.savefig('HW01_301_8.png')\nplt.show()",
    "detail": "HW.HW01_301",
    "documentation": {} },
    {
    "label": "plt.grid(color",
    "kind": 5,
    "importPath": "HW.HW01_301",
    "description": "HW.HW01_301",
    "peekOfCode": "plt.grid(color = 'gray', linestyle = '--', linewidth =
```

```

0.5)\nplt.savefig('HW01_301_8.png')\nplt.show()\n'''Problem 9.c'''\n#Use
Numpy's linspace to create an \"linspace\" array of 100 points similar to
MATLAB\ntheta = np.linspace(0, 2 * np.pi, 100)\n#Create the complex numbers
z_1 and z_2\n#J is represented as the imaginary number in Python\nz_1 =
np.exp(3j * theta)\nz_2 = 1j * np.exp(-2j * theta)\", \"detail\":
\"HW.HW01_301\", \"documentation\": {}, { \"label\": \"theta\",
\"kind\": 5, \"importPath\": \"HW.HW01_301\", \"description\":
\"HW.HW01_301\", \"peekOfCode\": \"theta = np.linspace(0, 2 * np.pi,
100)\n#Create the complex numbers z_1 and z_2\n#J is represented as the
imaginary number in Python\nz_1 = np.exp(3j * theta)\nz_2 = 1j * np.exp(-2j *
theta)\n#Create the figure\nplt.figure(figsize =
(12,6))\nplt.subplot(1,2,1)\nplt.plot(z_1.real, z_1.imag, label='z_1 =
e^(3j; ,r•ÆÇ ÇBÇF–FÆR,u & ÖFW&-2 Æ÷G2 öb ¥ó r''Ä \"detail\":
\"HW.HW01_301\", \"documentation\": {}, { \"label\": \"z_1\",
\"kind\": 5, \"importPath\": \"HW.HW01_301\", \"description\":
\"HW.HW01_301\", \"peekOfCode\": \"z_1 = np.exp(3j * theta)\nz_2 = 1j *
np.exp(-2j * theta)\n#Create the figure\nplt.figure(figsize =
(12,6))\nplt.subplot(1,2,1)\nplt.plot(z_1.real, z_1.imag, label='z_1 =
e^(3j; ,r•ÆÇ ÇBÇF–FÆR,u & ÖFW&-2 Æ÷G2 öb ¥ó r•ÆÇ ÇBÇ†Æ &VÄ,u&V Ä
Part')\nplt.ylabel(\"Imaginary Part\")\nplt.axhline(0,
color='black',linewidth=0.5, ls='--')\", \"detail\": \"HW.HW01_301\",
\"documentation\": {}, { \"label\": \"z_2\", \"kind\":
5, \"importPath\": \"HW.HW01_301\", \"description\": \"HW.HW01_301\",
\"peekOfCode\": \"z_2 = 1j * np.exp(-2j * theta)\n#Create the
figure\nplt.figure(figsize = (12,6))\nplt.subplot(1,2,1)\nplt.plot(z_1.real,
z_1.imag, label='z_1 = e^(3j; ,r•ÆÇ ÇBÇF–FÆR,u & ÖFW&-2 Æ÷G2 öb
z_1')\nplt.xlabel('Real Part')\nplt.ylabel(\"Imaginary
Part\")\nplt.axhline(0, color='black',linewidth=0.5, ls='--')\nplt.axvline(0,
color='black',linewidth=0.5, ls='--')\", \"detail\": \"HW.HW01_301\",
\"documentation\": {}, { \"label\": \"plt.figure(figsize\",
\"kind\": 5, \"importPath\": \"HW.HW01_301\", \"description\":
\"HW.HW01_301\", \"peekOfCode\": \"plt.figure(figsize =
(12,6))\nplt.subplot(1,2,1)\nplt.plot(z_1.real, z_1.imag, label='z_1 =
e^(3j; ,r•ÆÇ ÇBÇF–FÆR,u & ÖFW&-2 Æ÷G2 öb ¥ó r•ÆÇ ÇBÇ†Æ &VÄ,u&V Ä
Part')\nplt.ylabel(\"Imaginary Part\")\nplt.axhline(0,
color='black',linewidth=0.5, ls='--')\nplt.axvline(0,
color='black',linewidth=0.5, ls='--')\nplt.grid(color = 'gray', linestyle =
'--', linewidth = 0.5)\nplt.legend()\", \"detail\": \"HW.HW01_301\",
\"documentation\": {}, { \"label\": \"plt.grid(color\",
\"kind\": 5, \"importPath\": \"HW.HW01_301\", \"description\":
\"HW.HW01_301\", \"peekOfCode\": \"plt.grid(color = 'gray', linestyle =
'--', linewidth = 0.5)\nplt.legend()\nplt.subplot(1,2,2)\nplt.plot(z_2.real,
z_2.imag, label='z_2 = je^(-2j; ,r•ÆÇ ÇBÇF–FÆR,u & ÖFW&-2 Æ÷G2 öb
z_2')\nplt.xlabel('Real Part')\nplt.ylabel(\"Imaginary
Part\")\nplt.axhline(0, color='black',linewidth=0.5, ls='--')\nplt.axvline(0,
color='black',linewidth=0.5, ls='--')\nplt.grid(color = 'gray', linestyle =
'--', linewidth = 0.5)\", \"detail\": \"HW.HW01_301\",
\"documentation\": {}, { \"label\": \"plt.grid(color\",
\"kind\": 5, \"importPath\": \"HW.HW01_301\", \"description\":
\"HW.HW01_301\", \"peekOfCode\": \"plt.grid(color = 'gray', linestyle =
'--', linewidth = 0.5)\nplt.legend()\n#Save and show the
plots\nplt.savefig('HW01_301_9c.png')\nplt.show()\", \"detail\":
\"HW.HW01_301\", \"documentation\": {} }]}

```

## Examples\mycomet.py

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation
import FuncAnimation

def comet(x, y, *args, **kwargs):
    tf = 30
    fps = 30
    fig, ax = plt.subplots()
    ln, = plt.plot([], [], 'r.-', alpha=0.5,
    *args, **kwargs)
    dot, = plt.plot([], [], marker='o', markersize=10)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.grid()
    def init():
        ax.set_xlim(np.min(x)*1.01, np.max(x)*1.01)
        ax.set_ylim(np.min(y)*1.01, np.max(y)*1.01)
        return ln, dot
    def
    update(frame):
        i = int(frame*len(x)/(tf*fps))
        ln.set_data(x[:i
        + 1], y[:i + 1])
        dot.set_data(x[i], y[i])
        return ln, dot
    plt.close()
    return FuncAnimation(fig, update, frames=range(tf*fps),
    init_func=init, blit=True, interval=1000/fps)
```

## HW01\_301.py

```
import matplotlib.pyplot as plt
import numpy as np

# Problem 8
n = 50
roots = np.exp(1j * angles)
plt.figure(figsize = (10,10))
plt.plot(roots.real, roots.imag, 'o')
plt.title('Roots of Unity of n=50')
plt.xlabel('Real Part')
plt.ylabel('Imaginary Part')
plt.axhline(0, color='black', linewidth=0.5, ls='--')
plt.axvline(0, color='black', linewidth=0.5, ls='--')
plt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)
plt.savefig('HW01_301_8.png')
plt.show()

# Problem 9.c
# Use Numpy's linspace to create an "linspace" array of 100 points similar to MATLAB
theta = np.linspace(0, 2 * np.pi, 100)
# Create the complex numbers z_1 and z_2
z_1 = np.exp(3j * theta)
z_2 = 1j * np.exp(-2j * theta)
plt.figure(figsize = (12,6))
plt.subplot(1,2,1)
plt.plot(z_1.real, z_1.imag, label='z_1 = e^(3j;','r')
plt.title('Parameteric Plots of z_1')
plt.xlabel('Real Part')
plt.ylabel('Imaginary Part')
plt.axhline(0, color='black', linewidth=0.5, ls='--')
plt.axvline(0, color='black', linewidth=0.5, ls='--')
plt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)
plt.legend()
plt.subplot(1,2,2)
plt.plot(z_2.real, z_2.imag, label='z_2 = je^(-2j;','b')
plt.title('Parameteric Plots of z_2')
plt.xlabel('Real Part')
plt.ylabel('Imaginary Part')
plt.axhline(0, color='black', linewidth=0.5, ls='--')
plt.axvline(0, color='black', linewidth=0.5, ls='--')
plt.grid(color = 'gray', linestyle = '--', linewidth = 0.5)
plt.legend()
# Save and show the plots
plt.savefig('HW01_301_9c.png')
plt.show()
```

# Table of Contents

[.vscode\PythonImportHelper-v2-Completion.json](#)

..... [object Object]

[Examples\mycomet.py](#)

..... [object Object]

[HW\HW01\\_301.py](#)

..... [object Object]